

# Documentación MuMax3 AFNC

Víctor Costilla López

Versión 24.11

## Contenidos

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                       | <b>1</b>  |
| 1.1. Comentarios previos . . . . .                           | 1         |
| 1.2. Vista general del proyecto . . . . .                    | 2         |
| <b>2. TorqueFnAFNC</b>                                       | <b>4</b>  |
| 2.1. AddExchangeFieldAFNC . . . . .                          | 5         |
| 2.1.1. Variables para el cálculo del intercambio . . . . .   | 6         |
| 2.1.2. AddExchangeAFNCCell . . . . .                         | 6         |
| 2.1.3. AddExchangeAFNCll . . . . .                           | 7         |
| 2.1.4. AddDMIAFNC . . . . .                                  | 7         |
| 2.1.5. Compactación de las expresiones de la DMI . . . . .   | 9         |
| 2.1.6. Implementación de la DMI . . . . .                    | 10        |
| 2.2. AddAnisotropyFieldAFNC . . . . .                        | 10        |
| 2.2.1. Variables para el cálculo de la anisotropía . . . . . | 11        |
| 2.2.2. Conflictos entre anisotropías . . . . .               | 12        |
| 2.3. Cálculo de los torques (LL y STT) . . . . .             | 12        |
| <b>3. Resto de funciones en core</b>                         | <b>13</b> |
| <b>4. Archivo run y solvers</b>                              | <b>14</b> |
| 4.1. RK23 . . . . .                                          | 14        |
| 4.2. RK4 . . . . .                                           | 15        |
| <b>Referencias</b>                                           | <b>17</b> |

# 1. Introducción

En este documento se pueden encontrar las modificaciones hechas al código de `aleste2` [1], basado en `MuMax3` [2], con objetivo de introducir un tercer vector magnetización para estudiar los materiales antiferromagnéticos no colineales (AFNC). Las contribuciones descritas en este documento se pueden encontrar en mi repositorio [3].

## 1.1. Comentarios previos

El programa calcula la evolución de la magnetización reducida  $\vec{m}(\vec{r}, t)$  en cada punto (celda) del espacio. Para ello resuelve la ecuación diferencial

$$\frac{\partial \vec{m}}{\partial t} = \vec{\tau}. \quad (1.1)$$

Desde el punto de vista del funcionamiento del programa, este consta de dos partes. Un *solver* para obtener la solución numérica de dicha ecuación y una función *torque* para calcular el valor de  $\vec{\tau}$  en cada iteración del solver.

En realidad así funciona `MuMax3` original, en nuestro caso se resuelven tres ecuaciones

$$\frac{\partial \vec{m}_i}{\partial t} = \vec{\tau}_i, \quad (1.2)$$

donde ahora cada  $\vec{\tau}_i$  depende de las tres magnetizaciones. Se dedicará la [Sección 2](#) al torque, localizado en el archivo `src/github.com/mumax3/3/engine/ext_AF+NC_core.go`. La [Sección 3](#) tratará de otras funciones que se han modificado en el archivo `core` y en la [Sección 4](#) se hablará de los nuevos solvers para el sistema de ecuaciones diferenciales propuesto.

Para moverse dentro del proyecto es útil entender qué se ha programado usando CUDA o Go. Los archivos de Go son los que trabajan a “nivel más alto” mientras que en los CUDA se realizan las operaciones. A lo que me refiero es a que toda la estructura del programa está escrita en Go, es decir, todas las llamadas a funciones y sus relaciones entre ellas. Si se siguen sus dependencias, siempre acaban en `k_funcion_async`, que es la forma de llamar al archivo CUDA donde realmente está definida la función, cuyo nombre será `funcion.cu` y se encontrará en la carpeta `cuda`.

Para orientarse en el proyecto es útil recurrir a la [Subsección 1.2](#) y buscar la función que se quiere consultar. En ocasiones las dependencias no son claras, como en el archivo `AntiferroNC.go`, y con esta vista general no será suficiente. Para facilitar la búsqueda de dependencias al principio de cada sección se marca dentro de un recuadro verde el/los archivo/s relevantes para esa función.

Un último aspecto a resaltar es que se han mantenido todas las funcionalidades de `aleste2`. Esto se puede ver en los archivos `core` y `exchange`, ya que ambos usan la etiqueta `AF+NC`. Dichos archivos contienen todas las funciones de `aleste2` además de las nuevas. Se ha procedido de esta manera porque en `aleste2` ya hay dos vectores magnetización y se comparten muchas otras variables. Así no hace falta definir nuevos parámetros y complicar aún más la nomenclatura. Si se quiere simular antiferro, sólo hay que seleccionar un solver adecuado e igualar la tercera magnetización a cero.

## 1.2. Vista general del proyecto



Figura 1: Árbol de dependencias de la función `torque` en el archivo `core`. Tras cada función, su ruta desde `src/github.com/mumax/3`. Si no se indica ruta, está definida en `core`. En verde, archivos que no hizo falta modificar. En azul, archivos modificados. Y en rojo, archivos que faltan por implementar.

```

engine/ext_AF+NC_core.go
├─ MagnetizationAFNC
│  └─ MagnetizationAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.MagnetizationAFNC_async: cuda/ext_AFNC_MagnetizationAFNC.cu
├─ NeelnAFNC
│  └─ NeelnAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.NeelAFNC_async: cuda/ext_AFNC_Neel_n_AFNC.cu
├─ NeellAFNC
│  └─ NeellAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.NeellAFNC_async: cuda/ext_AFNC_Neel_l_AFNC.cu
├─ GMagnetizationAFNC
│  └─ GMagnetizationAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.GMagnetizationAFNC_async: cuda/ext_AFNC_GMagnetizationAFNC.cu
├─ GNeelAFNC
│  └─ GNeelAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.GNeelAFNC_async: cuda/ext_AFNC_GNeel_n_AFNC.cu
├─ GNeellAFNC
│  └─ GNeellAFNC: cuda/ext_AFNC_AntiferroNC.go
│     └─ k.GNeellAFNC_async: cuda/ext_AFNC_GNeel_l_AFNC.cu
├─ SetMFull11
├─ SetMFull12
└─ SetMFull13

```

Figura 2: Resto de funciones modificados dentro del archivo core.

```

engine/
├─ ext_AF+NC_core.go
├─ ext_AF+NC_exchange.go
├─ ext_AFNC_rk23.go
├─ ext_AFNC_rk4.go
└─ run.go

```

Figura 3: Archivos modificadas en el directorio **engine/**

## 2. TorqueFnAFNC

```
engine/ext_AF+NC_core.go
```

La función torque devuelve tres campos vectoriales `dsti` (con  $i=1,2,3$ ) que son los valores de  $\vec{\tau}_i$  para cada celda del material y para un tiempo concreto. Se calculan considerando tres torques: el de Landau-Lifshitz ( $\vec{\tau}_{LL}$ ) y los dos de transferencia de spin, Zhang-Li ( $\vec{\tau}_{ZL}$ ) e Slonczewski ( $\vec{\tau}_{SL}$ ) [2]. De estos tres, el único que depende explícitamente del campo magnético es el primero, cuya expresión es

$$\vec{\tau}_{LL,i} = \frac{\gamma_{LL,i}}{1 + \alpha_i^2} \left( \vec{m}_i \times \vec{B}_{\text{eff},i} + \alpha_i (\vec{m}_i \times \vec{B}_{\text{eff},i}) \right), \quad (2.1)$$

donde  $\gamma_{LL,i}$  son los factores giromagnéticos para cada subred,  $\alpha_i$  las constantes de amortiguación y  $\vec{B}_{\text{eff},i}$  los campos magnéticos efectivos. Estos últimos son suma de los campos:

1. Externo,  $\vec{B}_{\text{ext}}$
2. Desmagnetización (magnetostático),  $\vec{B}_{\text{demag}}$
3. Intercambio de Heisenberg,  $\vec{B}_{\text{exch}}$  [Secciones 2.1.2 y 2.1.3]
4. Intercambio de Dzyaloshinskii-Moriya (DMI),  $\vec{B}_{\text{dmi}}$  [Sección 2.1.4]
5. Anisotropía magneto-cristalina,  $\vec{B}_{\text{anis}}$  [Sección 2.2]
6. Térmico,  $\vec{B}_{\text{therm}}$

Sólo se han modificado ambos campos de intercambio y el de anisotropía. Los primeros, por nuevas interacciones que surgen entre las subredes. Y el último, por considerar anisotropías tetragonal y hexagonal a mayores de las uniaxial y cúbica ya implementadas.

Aunque las expresiones del resto de campos no se han modificado, si se han hecho cambios en *core* cuando se llaman. Se enumeran a continuación.

---

*Comentario* (Sobre el uso de los `dsti`). Al comienzo de esta sección se ha atribuido a cada `dsti` un  $\vec{\tau}_i$ , pero en el cálculo de campos se les atribuye  $\vec{B}_{\text{eff},i}$ . Esto sucede porque dichos objetos se reutilizan para el cómputo de ambas magnitudes dentro de la función torque. Primero se usan para obtener el campo efectivo en cada celda, después se calcula  $\vec{\tau}_{LL,i}(\vec{m}_i, \vec{B}_{\text{eff},i})$  con ellos como argumento. Los demás torques no requieren los campos efectivos así que el valor de  $\vec{\tau}_{LL,i}$  se almacena sobrescribiendo  $\vec{B}_{\text{eff},i}$  en `dsti`.

---

### Campo externo

Se ejecuta `B_ext.AddTo(dsti)` para cada subred. Simplemente añade el valor del campo externo aplicado para cada celda del espacio en un tiempo dado.

### Campo de desmagnetización

Se ejecuta

```
SetDemagField(dst1)
data.Copy(dst2,dst1)
data.Copy(dst3,dst1)
```

Con esto se consigue tener el mismo campo en todas las subredes. Se calcula sólo una vez y se copia para cada una de ellas. Podemos hacerlo de esta manera porque es el primer sumando del campo efectivo, y al copiar no perdemos la información de las otras subredes. Esta misma idea se podría aplicar para el externo, pero sumar campos vectoriales es más sencillo que calcular este.

## Campo térmico

Este campo se introduce mediante `B_therm.AddToAFNC(dst1, dst2, dst3)`. Donde la función `AddToAFNC(dst1, dst2, dst3)` llama a `updateAFNC(i)` con  $i=1,2,3$ . Esta última función es idéntica a la versión de `aleste2`, pero incluye un tercer valor de  $i$ . Dicha variable solo determina que  $M_{\text{sat},i}$  y  $\alpha_i$  se utilizan en el cómputo del campo.

## 2.1. AddExchangeFieldAFNC

engine/ext\_AF+NC\_exchange.go

La expresión para la densidad de energía de intercambio [4] (siguiendo el convenio de Einstein) es

$$\varepsilon_{\text{exch}} = A_{ii}\|\vec{\nabla}\vec{m}_i\|^2 + A_{ij}\langle\vec{\nabla}\vec{m}_i, \vec{\nabla}\vec{m}_j\rangle + B_{ij}\vec{m}_i\vec{m}_j \quad (i \neq j), \quad (2.2)$$

donde  $A_{ij}$  y  $B_{ij}$  son constantes que regulan las interacciones entre magnetizaciones de distintas celdas y de la misma celda respectivamente. De esta definición se obtienen los siguientes campos de intercambio

$$\vec{B}_{\text{exch},i} = \underbrace{\frac{B_{ij}}{M_{s,i}}\vec{m}_j}_{\text{ExchCell}} - \underbrace{\frac{2A_{ii}}{M_{s,i}}\nabla^2\vec{m}_i}_{\text{Exch}} - \underbrace{\frac{A_{ij}}{M_{s,i}}\nabla^2\vec{m}_j}_{\text{Exchl1}} \quad (i \neq j), \quad (2.3)$$

con  $M_{s,i}$  las magnetizaciones de saturación para cada subred. Esta ecuación se ha implementado separando cada sumando en una función distinta. El segundo de ellos corresponde a la función `AddExchange` de MuMax3 original, el resto son nuevas.

También se incluye en esta función la contribución de DMI, su densidad de energía [5] es

$$\varepsilon_{\text{dmi},i} = D_i \left[ (\vec{m}_i \cdot \vec{u}_n) \vec{\nabla}\vec{m}_i - \vec{m}_i \cdot \vec{\nabla}(\vec{m}_i \cdot \vec{u}_n) \right], \quad (2.4)$$

donde  $D_i$  son constantes definidas para cada subred. Y  $\vec{u}_n$  es un vector unitario perpendicular a la interfaz entre las láminas de acople espín-órbita fuerte y ferromagnética. En el código se ha escogido la dirección  $z$ . El campo correspondiente es

$$\vec{B}_{\text{dmi},i} = \frac{2D_i}{M_{s,i}} \left[ \vec{\nabla}(\vec{m}_i \cdot \vec{u}_n) - (\vec{\nabla} \cdot \vec{m}_i) \vec{u}_n \right]. \quad (2.5)$$

Existe otro tipo de DMI que no es interfacial, en el código se llama `DMIBulk` y no se ha implementado. Tiene asociadas otras constantes  $D_{\text{bulk},i}$ . La función `AddExchangeFieldAFNC` añade todos estos campos a sus correspondientes `dsti` de la siguiente forma<sup>1</sup>

```
switch
case !inter1 && !bulk1:
    cuda.AddExchange(dst1)
    cuda.AddExchange(dst2)
    cuda.AddExchange(dst3)
    cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
    cuda.AddExchangeAFNCll(dst1, dst2, dst3)
case inter1 && !bulk1:
    cuda.AddDMIAFNC(dst1, dst2, dst3)
    cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
case bulk1 && !inter1:
```

---

<sup>1</sup>Este texto es una especie de pseudocódigo, la sintaxis no es correcta y faltan argumentos. Esto será cierto para la mayoría de “códigos” en la documentación.

```

        cuda.AddDMIBulk(dst1)
        cuda.AddDMIBulk(dst2)
        cuda.AddDMIBulk(dst3)
        cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
        cuda.AddExchangeAFNCll(dst1, dst2, dst3)
    case inter1 && bulk1:
        util.Fatal

```

donde las variables `inter1` y `bulk1` comprueban si las constantes  $D_1$  y  $D_{\text{bulk},1}$  son distintas de cero, respectivamente. Lo hacen para la primera subred, así que sus constantes deciden que interacciones se calcularán. No se pueden dar ambas DMI a la vez, como se puede ver en el pseudocódigo anterior, del cual solo nos interesan los primeros casos: sin y con DMI interfacial, sin DMI en volumen.

### 2.1.1. Variables para el cálculo del intercambio

La correspondencia entre las constantes de la Ecuación 2.3 y las variables usadas es

$$B_{ij} \rightarrow \text{Bexij} \quad A_{ii} \rightarrow \text{Aexi} \rightarrow \text{lex2i} \quad A_{ij} \rightarrow \text{Aexllij} \rightarrow \text{lexllij}, \quad (2.6)$$

donde ambos `lex` son `exchParam` dentro del código, y estos parámetros son los que se introducen como argumento a las funciones. El carácter 2 no es una errata, se ha heredado del código original de MuMax3.

Sucede que  $A_{ij} = A_{ji}$ , por tanto sólo hay definidos tres `lexllij`. En el caso de `Bexij` sí están definidas todas las combinaciones, aunque creemos que no es necesario, pero también se ha heredado y por conveniencia no se cambió (solo hace falta igualar sus valores al ejecutar el programa). De manera similar, para DMI se usan

$$D_i \rightarrow \text{Dindi} \rightarrow \text{din2i} \quad D_{\text{bulk},i} \rightarrow \text{Dbulki} \rightarrow \text{dbulk2i}. \quad (2.7)$$

Para las magnetizaciones<sup>2</sup> y magnetizaciones de saturación de cada subred se usan las variables

$$\vec{m}_i \rightarrow \text{Mi} \quad M_{s,i} \rightarrow \text{Msati}, \quad (2.8)$$

que se utilizarán en el cálculo de los demás campos.

### 2.1.2. AddExchangeAFNCCell

`cuda/ext_AFNC_AntiferroNC_core.go, cuda/ext_AFNC_AddExchangeAFNCCell`

El primer sumando de la Ecuación 2.3 calcula el campo debido a la magnetización de las otras subredes evaluadas en la misma celda en que se está calculando. Se ha definido la función `AddExchangeAFNCCell`, que añade a cada `dsti`

```

dst1x[i] += invMs1*bex12*m2x[i] + invMs1*bex13*m3x[i];
dst1y[i] += invMs1*bex12*m2y[i] + invMs1*bex13*m3y[i];
dst1z[i] += invMs1*bex12*m2z[i] + invMs1*bex13*m3z[i];

```

Donde `invMs1` es simplemente  $1/M_{s,i}$ . Solo se han escrito los términos para la primera subred, es fácil ver la forma para las demás atendiendo a la Ecuación 2.3. Se ha abandonado la notación de  $i$ -ésima subred porque en este código dicho índice se reserva para designar la celda en que se está trabajando<sup>3</sup>

<sup>2</sup>Las variables `Mi` son del tipo `magnetization`, definido en `engine/magnetization.go`, que están equipadas con funciones para obtener sus componentes en cada celda.

<sup>3</sup>No realmente, este índice tiene que ver con “hilos” y “bloques” de la GPU. No se va a entrar en esas consideraciones.

### 2.1.3. AddExchangeAFNCII

cuda/ext\_AFNC\_AntiferroNC.go, cuda/exchange.cu

El tercer sumando de la Ecuación 2.3 relaciona las magnetizaciones de distintas subredes y de diferentes celdas, en particular, de los vecinos ortogonales. La forma funcional de este sumando es casi idéntica al segundo, que ya está implementado en MuMax3. Por lo tanto, con solo cambiar los argumentos de la función original se obtiene el resultado esperado

```
//Sublattice 1
k_addexchange_async(dst1, m2, ms1, llex12)
k_addexchange_async(dst1, m3, ms1, llex13)
//Sublattice 2
k_addexchange_async(dst2, m1, ms2, llex12)
k_addexchange_async(dst2, m3, ms2, llex23)
//Sublattice 3
k_addexchange_async(dst3, m1, ms3, llex13)
k_addexchange_async(dst3, m2, ms3, llex23)
```

En cada llamada de la función `addexchange` se computa la divergencia con las magnetizaciones de otra subred, utilizando la constante  $A_{ij}$  adecuada y con  $M_{s,i}$  de la red para la cual se calcula el campo.

### 2.1.4. AddDMIAFNC

cuda/ext\_AFNC\_dmi.go, cuda/ext\_AFNC\_dmi.cu

Una versión más explícita de la Ecuación 2.5 es

$$\vec{B}_{\text{dmi},i} = \frac{2D_i}{M_{s,i}} (\partial_x m_{i,z}, \partial_y m_{i,z}, -\partial_x m_{i,x} - \partial_y m_{i,y}). \quad (2.9)$$

Para calcular derivadas se necesita conocer la magnetización de los vecinos, pero si alguno de ellos se encuentra fuera de los límites de la muestra, habrá que extrapolarlo. Para ello se imponen las siguientes condiciones de contorno

$$\vec{m}_i \times \left[ \left( 2A_{ii}(\vec{\nabla}\vec{m}_i) + A_{ij}(\vec{\nabla}\vec{m}_j) \right) \cdot \hat{n} + D_i \vec{m}_i \times (\hat{n} \times \vec{u}_z) \right] = 0 \quad (i \neq j), \quad (2.10)$$

donde  $\hat{n}$  es un vector unitario perpendicular a la superficie del borde que se está considerando. Se van a imponer unas condiciones menos generales, en particular, que todo el corchete de la Ecuación 2.10 sea nulo. Con esas condiciones se obtienen los siguientes resultados:



## Bordes X

$$\begin{aligned} \partial_x m_{1,x} = & - \frac{1}{2A_{11} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}}\right) - \frac{\left(A_{13} - \frac{A_{12}A_{23}}{2A_{22}}\right)^2}{2A_{33} \left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}}\right)}} \left[ D_1 m_{1,z} + \left( \frac{A_{23} \left( A_{13} - \frac{A_{12}A_{23}}{2A_{22}} \right)}{4A_{22}A_{33} \left( 1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)} - \frac{A_{12}}{2A_{22}} \right) D_2 m_{2,z} \right. \\ & \left. - \left( \frac{A_{13} - \frac{A_{12}A_{23}}{2A_{22}}}{2A_{33} \left( 1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)} \right) D_3 m_{3,z} \right], \end{aligned} \quad (2.11)$$

$$\begin{aligned} \partial_x m_{2,x} = & - \frac{1}{2A_{22} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}}\right) - \frac{\left(A_{23} - \frac{A_{12}A_{13}}{2A_{11}}\right)^2}{2A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}}\right)}} \left[ \left( \frac{A_{13} \left( A_{23} - \frac{A_{12}A_{13}}{2A_{11}} \right)}{4A_{11}A_{33} \left( 1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right)} - \frac{A_{12}}{2A_{11}} \right) D_1 m_{1,z} + D_2 m_{2,z} \right. \\ & \left. - \left( \frac{A_{23} - \frac{A_{12}A_{13}}{2A_{11}}}{2A_{33} \left( 1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right)} \right) D_3 m_{3,z} \right], \end{aligned} \quad (2.12)$$

$$\begin{aligned} \partial_x m_{3,x} = & - \frac{1}{2A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}}\right) - \frac{\left(A_{23} - \frac{A_{12}A_{13}}{2A_{11}}\right)^2}{2A_{22} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}}\right)}} \left[ \left( \frac{A_{12} \left( A_{23} - \frac{A_{12}A_{13}}{2A_{11}} \right)}{4A_{11}A_{22} \left( 1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right)} - \frac{A_{13}}{2A_{11}} \right) D_1 m_{1,z} \right. \\ & \left. - \left( \frac{A_{23} - \frac{A_{12}A_{13}}{2A_{11}}}{2A_{11} \left( 1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right)} \right) D_2 m_{2,z} + D_3 m_{3,z} \right]. \end{aligned} \quad (2.13)$$

$$\partial_x m_{1,y} = \partial_x m_{2,y} = \partial_x m_{3,y} = 0, \quad (2.14)$$

$$\partial_x m_{1,z} = -\partial_x m_{1,x} \quad [x \leftrightarrow z], \quad (2.15)$$

$$\partial_x m_{2,z} = -\partial_x m_{2,x} \quad [x \leftrightarrow z], \quad (2.16)$$

$$\partial_x m_{3,z} = -\partial_x m_{3,x} \quad [x \leftrightarrow z]. \quad (2.17)$$

Los corchetes en las tres últimas ecuaciones significan sustituir las componentes  $x$  de  $m_i$  por sus componentes  $z$  en el lado derecho de la igualdad.

## Bordes Y

$$\partial_y m_{1,x} = \partial_y m_{2,y} = \partial_y m_{3,y} = 0, \quad (2.18)$$

$$\partial_y m_{1,y} = \partial_x m_{1,x}, \quad (2.19)$$

$$\partial_y m_{2,y} = \partial_x m_{2,x}, \quad (2.20)$$

$$\partial_y m_{3,y} = \partial_x m_{3,x}, \quad (2.21)$$

$$\partial_y m_{1,z} = -\partial_x m_{1,x} \quad [x \leftrightarrow z], \quad (2.22)$$

$$\partial_y m_{2,z} = -\partial_x m_{2,x} \quad [x \leftrightarrow z], \quad (2.23)$$

$$\partial_y m_{3,z} = -\partial_x m_{3,x} \quad [x \leftrightarrow z]. \quad (2.24)$$

## Bordes Z

$$\partial_z m_{1,x} = \partial_z m_{2,x} = \partial_z m_{3,x} = 0, \quad (2.25)$$

$$\partial_z m_{1,y} = \partial_z m_{2,y} = \partial_z m_{3,y} = 0, \quad (2.26)$$

$$\partial_z m_{1,z} = \partial_z m_{2,z} = \partial_z m_{3,z} = 0. \quad (2.27)$$

Donde las ecuaciones de los bordes  $z$  además de 2.14 y 2.18 son ciertas si se cumple

$$\left(1 - \frac{\left(A_{13} - \frac{A_{12}A_{23}}{2A_{22}}\right)^2}{4A_{11}A_{33}\left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}}\right)\left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}}\right)}\right) \neq 0. \quad (2.28)$$

### 2.1.5. Compactación de las expresiones de la DMI

`cuda/dmiafnc.h`

Aprovechando la repetición de ciertos términos en las ecuaciones anteriores, se han implementado definiendo funciones intermedias que calculen cada uno de ellos. Por conveniencia, se prescinde de un índice y nos referiremos a las constantes  $A$  siguiendo este criterio

$$\begin{aligned} A_{11} &\rightarrow A_1 & A_{22} &\rightarrow A_2 & A_{33} &\rightarrow A_3, \\ A_{12} &\rightarrow A_4 & A_{13} &\rightarrow A_5 & A_{23} &\rightarrow A_6. \end{aligned} \quad (2.29)$$

Las funciones intermedias son las siguientes

$$\text{mid1} = A_5 - \frac{A_4A_6}{2A_2}, \quad (2.30)$$

$$\text{mid2} = 2A_3 \left(1 - \frac{(A_6)^2}{4A_2A_3}\right), \quad (2.31)$$

$$\text{pref} = \frac{1}{2A_1 \left(1 - \frac{(A_4)^2}{4A_1A_2}\right) - \frac{\left(A_5 - \frac{A_4A_6}{2A_2}\right)^2}{2A_3 \left(1 - \frac{(A_6)^2}{4A_2A_3}\right)}} = \frac{1}{2A_1 \left(1 - \frac{(A_4)^2}{4A_1A_2}\right) - \frac{(\text{mid1})^2}{\text{mid2}}}, \quad (2.32)$$

$$\text{coef1} = \frac{A_6 \left(A_5 - \frac{A_4A_6}{2A_2}\right)}{4A_2A_3 \left(1 - \frac{(A_6)^2}{4A_2A_3}\right)} - \frac{A_4}{2A_2} = \frac{A_6 \cdot \text{mid1}}{2A_2 \cdot \text{mid2}} - \frac{A_4}{2A_2}, \quad (2.33)$$

$$\text{coef2} = -\frac{A_5 - \frac{A_4A_6}{2A_2}}{2A_3 \left(1 - \frac{(A_6)^2}{4A_2A_3}\right)} = -\frac{\text{mid1}}{\text{mid2}}. \quad (2.34)$$

Todas estas funciones están definidas en `cuda/dmiafnc.h`. Estas funciones son simplemente otra forma de escribir la condición de contorno  $\partial_x m_{1,x}$

$$\partial_x m_{1,x} = -\text{pref} \cdot (D_1 m_{1,z} + \text{coef1} \cdot D_2 m_{2,z} + \text{coef2} \cdot D_3 m_{3,z}). \quad (2.35)$$

Lo interesante es que las otras dos condiciones que necesitamos saber  $\partial_x m_{2,x}$  y  $\partial_x m_{3,x}$  se pueden escribir de manera muy similar si cambiamos el orden de las variables  $A$ . Si entendemos estas funciones como

```
pref(A1,A2,A3,A4,A5,A6)
coef1(A1,A2,A3,A4,A5,A6)
coef2(A1,A2,A3,A4,A5,A6)
```

Y para cada subred introducimos las constantes en los argumentos siguiendo esta tabla

|       | $\partial_x m_{1,x}$ | $\partial_x m_{2,x}$ | $\partial_x m_{3,x}$ |
|-------|----------------------|----------------------|----------------------|
| pref  | 123456               | 213465               | 312564               |
| coef1 | 123456               | 213465               | 312564               |
| coef2 | 123456               | 213465               | 312564               |

Entonces las otras condiciones de contorno toman la forma

$$\partial_x m_{2,x} = -\text{pref} \cdot (\text{coef1} \cdot D_1 m_{1,z} + D_2 m_{2,z} + \text{coef2} \cdot D_3 m_{3,z}), \quad (2.36)$$

$$\partial_x m_{3,x} = -\text{pref} \cdot (\text{coef1} \cdot D_1 m_{1,z} + \text{coef2} \cdot D_2 m_{2,z} + D_3 m_{3,z}). \quad (2.37)$$

Como el resto de condiciones son las mismas cambiando  $m_{i,z} \rightarrow m_{i,x}$  o nulas, con esto hemos compactado todas las expresiones. De manera resumida:

|       | $D_1$ | $D_2$ | $D_3$ | Orden  |
|-------|-------|-------|-------|--------|
| $m_1$ | 1     | $C_1$ | $C_2$ | 123456 |
| $m_2$ | $C_1$ | 1     | $C_2$ | 213465 |
| $m_3$ | $C_1$ | $C_2$ | 1     | 312564 |

### 2.1.6. Implementación de la DMI

Como se puede ver en la estructura de la función `AddExchangeFieldAFNC` en la Sección 2.1, en el caso de DMI interfacial no se llama a `AddExchange`. Esto es porque dentro de la DMI también se calcula el intercambio usual, ya que estamos calculando el valor de las magnetizaciones vecinas con las condiciones de contorno (si son nulas). Y con estas magnetizaciones vecinas se computan las derivadas direccionales necesarias para ambas interacciones.

## 2.2. AddAnisotropyFieldAFNC

`engine/ext_AF+NC_core.go, engine/anisotropy.go`

La función `AddAnisotropyFieldAFNC` tiene como argumentos los tres campos `dsti` y añade el valor del campo de anisotropía magnetocristalina a cada uno. Se consideran los siguientes tipos de anisotropía, implementados en las funciones:

- Uniaxial. `addUniaxialAnisotropyFrom`
- ◆ Tetragonal. `addTetragonalAnisotropyFrom`
- ★ Hexagonal. `addHexagonalAnisotropyFrom`
- Cúbica. `addCubicAnisotropyFrom`

Que toman como argumentos ciertas constantes que definiremos a continuación además de las variables `dsti` donde se almacena la suma de los campos efectivos para cada subred y las propias magnetizaciones  $M_i$ . Estas funciones se encuentran definidas en `engine/anisotropy.go`

y en sus respectivos archivos `.cu` en `cuda/`, cuyos nombres se pueden consultar en el árbol de dependencias. Tanto la uniaxial como la cúbica no se han modificado, sus expresiones se pueden encontrar en la documentación de `Mumax3` [2]. La anisotropía cúbica es incompatible con el resto, pero el resto comparten términos [6]. Aunque no es físicamente correcto, se ha asignado a cada tipo de anisotropía únicamente los sumandos que son exclusivos de esta. Por lo tanto, el campo que se añade tiene la siguiente forma

$$\vec{B}_{\text{anis},i} = \vec{B}_{\text{uniax},i} + \vec{B}_{\text{tetrag},i} + \vec{B}_{\text{hex},i} + \vec{B}_{\text{cubic},i}, \quad (2.38)$$

en donde varios de los sumandos pueden ser nulos reproduciendo así los campos deseados, más información al respecto en la Subsección 2.2.2. Los subíndices  $i = 1, 2, 3$  indican la subred en la que se calcula el campo, y como se puede ver, en esta sección no se mezclan las subredes. Las expresiones de los campos correspondientes a la Ecuación 2.38, tal como están definidos en el código son

$$\vec{B}_{\text{uniax},i} = \frac{2K_{1,i}}{M_{s,i}}(\vec{m}_i \cdot \vec{u}_z)\vec{u}_z + \frac{4K_{2,i}}{M_{s,i}}(\vec{m}_i \cdot \vec{u}_z)^3\vec{u}_z, \quad (2.39)$$

$$\vec{B}_{\text{tetrag},i} = -\frac{2K_{2b,i}}{M_{s,i}}m_x m_y (m_x \cdot \vec{u}_y + m_y \cdot \vec{u}_x), \quad (2.40)$$

$$\vec{B}_{\text{hex},i} = \frac{6K_{3,i}}{M_{s,i}}(\vec{m}_i \cdot \vec{u}_k)^5\vec{u}_k + \frac{6K_{3b,i}}{M_{s,i}}[5m_x m_y (m_x^3 \cdot \vec{u}_x + m_y^3 \cdot \vec{u}_y) - 10m_x^2 m_y^2 (m_x \cdot \vec{u}_y + m_y \cdot \vec{u}_x) + m_x^5 \cdot \vec{u}_y + m_y^5 \cdot \vec{u}_x]. \quad (2.41)$$

Los términos que acompañan a potencias de  $m_z$  solo definen una dirección privilegiada (uniaxial), por lo que la única diferencia entre los sumandos con  $K_1, K_2, K_3$  es la intensidad de ese campo. En cualquier caso, si se quieren utilizar esos términos se da la opción para ello.

En cada llamada de `AddAnisotropyFieldAFNC` se llaman a todos los campos, entonces para controlar qué sumandos se consideran hay que tomar nulas ciertas constantes  $K_{ij}$ . Por defecto todas lo son, así que para recuperar las expresiones correctas de cada campo hay que asignar valores a estas constantes de acuerdo a la siguiente tabla.

|              | $K_1$   | $K_2$   | $K_{2b}$ | $K_3$   | $K_{3b}$ |
|--------------|---------|---------|----------|---------|----------|
| Uniaxial •   | ✓       | ✓       | ✗        | ✓       | ✗        |
| Tetragonal ♦ | ✓       | ✓       | ✓        | ✓       | ✗        |
| Hexagonal ★  | ✓       | ✓       | ✗        | ✓       | ✓        |
|              | Orden 1 | Orden 2 |          | Orden 3 |          |

Los signos ✓ indican que la constante aparece en la expresión del campo para ese tipo de anisotropía y los signos ✗ indican lo contrario. Por lo tanto, si queremos considerar un tipo en concreto, las constantes con ✗ necesariamente han de ser cero y el resto pueden tener un valor distinto de cero.

Cada una de las funciones encargadas de calcular los campos de de anisotropía solo se calculan si alguna de las constantes de las que depende no es nula. A pesar de que sin esta condición se obtienen los resultados correctos, de esta manera se evita calcular multitud de campos que van a ser nulos. Ya que el cómputo se realiza en cada celda de la muestra, para cada subred y en cada instante temporal.

### 2.2.1. Variables para el cálculo de la anisotropía

La correspondencia entre las constantes de la Ecuación 2.39 y las variables usadas es

$$K_{ij} \rightarrow \text{Kuij} \quad \vec{u}_{z,i} \rightarrow \text{AnisUi} \quad \vec{u}_{x,i} \rightarrow \text{AnisUib}, \quad (2.42)$$

donde los subíndices  $(i, j) = (\text{subred}, \text{orden})$ . Cabe mencionar que la constante con orden  $j$  acompaña a un término de grado  $2j - 1$  en  $m$  (En las expresiones energéticas, acompañan a grado  $2j$  [6]). De manera similar, para el cálculo del campo de anisotropía cúbica se utilizan

$$K_{c,ij} \rightarrow \text{Kc}ij \quad \vec{u}_{z,i} \rightarrow \text{AnisC1i} \quad \vec{u}_{x,i} \rightarrow \text{AnisC2i}. \quad (2.43)$$

Donde las direcciones que formarán los ejes para la anisotropía cúbica se llaman **C1i** y **C2i** en vez de **Ui** y **Uib**. En ambos tipos sólo se proporcionan los ejes  $\hat{z}$  y  $\hat{x}$  porque el tercero se calcula como producto vectorial de los anteriores ( $\hat{y} = \hat{z} \times \hat{x}$ ). De momento no hay implementada diferencia entre los sistemas de referencia laboratorio y cristalográfico. Si se implementa en un futuro, las direcciones de anisotropía se darán en función de solo uno de ellos.

### 2.2.2. Conflictos entre anisotropías

Ya que físicamente un sistema solo tiene un tipo de anisotropía de los que hemos definido se han añadido casos conflictivos que detienen la ejecución del programa. Para identificarlos se han implementado definiendo las siguientes variables:

```
haveUniaxial := Ku11.nonZero() || Ku21.nonZero()
haveCubic := Kc11.nonZero() || Kc21.nonZero() || Kc31.nonZero()
haveTetrag := Ku21b.nonZero()
haveHexag := Ku31.nonZero() || Ku31b.nonZero()
```

Se toma como dominante la primera subred, de ahí el segundo índice de las variables es 1. Como las expresiones para los casos tetragonal y hexagonal comparten sumandos con el caso uniaxial no hay problema si **haveUniaxial** toma el valor verdadero a la vez que **haveTetrag** o **haveHexag**, pero estas son incompatibles entre ellas.

```
if haveTetrag && haveHexag {
    util.Fatal("Cannot have both tetragonal and hexagonal anisotropy")
}
```

De manera similar el caso cúbico es incompatible con el resto, y aprovechando que todas comparten términos con el caso uniaxial menos la cúbica, esta condición se puede expresar como

```
if haveUniaxial && haveCubic {
    util.Fatal("Cannot have both uniaxial and cubic anisotropy")
}
```

## 2.3. Cálculo de los torques (LL y STT)

Las últimas secciones se han dedicado a explicar los sumandos de los campos magnéticos efectivos de cada subred, ya que el torque de Landau-Lifshitz ( $\vec{\tau}_{LL}$ ) necesita ese campo para calcularse. Pero como se dijo en la Sección 2 también existen dos torques más que no necesitan este campo. Hasta ahora no se han explicado las funciones encargadas de calcular estos torques, estas son:

- $\vec{\tau}_{LL}$ :
  - Si hay precesión **LLTorque**
  - Si no hay precesión **LLNoPrecess**
- $\vec{\tau}_{ZL} + \vec{\tau}_{SL}$ : **AddSTTorqueAFNC**

Las dos primeras funciones no se han modificado, pero para calcular el torque en cada subred, se llaman por triplicado, aplicándose sobre cada `dsti` por separado. La función `AddSTTorqueAFNC` mantiene intacta la estructura de `aleste2` pero llamando tres veces a cada función involucrada en el cálculo de los torques de transferencia de espín, utilizando las variables de una subred cada vez. Estas funciones son:

- `AddZhangLiTorque`
- `AddSlonczewskiTorque2`
- `AddSlonczewskiTorque2LLB`

También se han triplicado las variables dependientes de cada subred necesarias para el cómputo de estos torques (e.g. tres constantes de amortiguación  $\alpha_i$ ), es fácil verlo en las dependencias de estas funciones en el código.

### 3. Resto de funciones en core

Ya se han descrito todas las funciones encargadas de calcular los torques en cada iteración temporal. Pero se han omitido algunas funciones también definidas en el archivo `core` que o bien no aparecen directamente en el cálculo de las magnitudes anteriores o que apenas han sido modificadas. Por completitud, a continuación se encuentran los cambios realizados/definiciones de estas funciones.

#### MagnetizationAFNC

`cuda/ext_AFNC_MagnetizationAFNC.cu, cuda/ext_AFNC_GMagnetizationAFNC.cu`

Definición del vector  $\vec{m} = (\vec{m}_1 + \vec{m}_2 + \vec{m}_3)/3$ , magnetización reducida total. Al trabajar con un sistema antiefromagnético ha de ser nula (a excepción de errores de redondeo).

#### NeellAFNC

`cuda/ext_AFNC_NeellAFNC.cu, cuda/ext_AFNC_GNeellAFNC.cu`

Definición del vector  $\vec{l} = (\vec{m}_1 - \vec{m}_2 + \vec{m}_3)/3$ , que junto a  $\vec{m}$  y  $\vec{n}$  definido a continuación proporcionan una base adaptada al sistema antiefromagnético, para poder extraer información relevante.

#### NeelnAFNC

`cuda/ext_AFNC_NeelnAFNC.cu, cuda/ext_AFNC_GNeelnAFNC.cu`

Definición del vector  $\vec{n} = (\vec{m}_1 + \vec{m}_2 - \vec{m}_3)/3$

Estas tres funciones también tienen versiones en las que cada magnetización reducida se divide por el factor giromagnético ( $\vec{m}_i \rightarrow \vec{m}_i/\gamma_i$ ) de cada subred. En principio son todos iguales, pero se permite definirlos individualmente. Las funciones llevan el mismo nombre precedido por una G.

## SetMFull3

Función que inicializa la magnetización de la tercera subred en todo el material, escalando su valor en cada celda de acuerdo a su volumen. El código es idéntico al de `aleste2` (y también `MuMax3`) pero usando la magnetización reducida y de saturación correspondiente a esta subred.

## 4. Archivo run y solvers

```
engine/run.go
```

El archivo `run.go` se encarga de seleccionar el solver e inicializarlo. Para ello se asigna un ID a cada tipo y en el archivo `.mx3` se selecciona mediante

```
SetSolver(ID)
```

Como en nuestro caso hay que resolver el sistema de tres ecuaciones diferenciales  $\frac{\partial \vec{m}_i}{\partial t} = \vec{\tau}_i$ , simplemente tendremos que repetir el código existente para una subred y que genere el valor de  $\vec{m}_i$  para cada subred en el siguiente paso temporal. Se han adaptado únicamente los solvers Runge-Kutta de orden 23 y 4, con los siguientes IDs

```
ANTIFERRORK4   = 100
ANTIFERRORK23  = 101
ANTIFERRONCRK4 = 300
ANTIFERRONCRK23 = 301
```

Los valores 100 y 101 sirven para recuperar el caso de antiefromagnético colineal mientras que 300 y 301 tienen en cuenta las tres subredes.

Dentro de la función `SetSolver`, en un bloque `switch` para los tipos, se han añadido las llamadas a los nuevos solvers.

```
switch typ {
default:
    util.Fatalf("SetSolver: unknown solver type: %v", typ)
...
case ANTIFERRONCRK4:
    stepper = new(AntiferroNCRK4)
    AFf = true
case ANTIFERRONCRK23:
    stepper = new(AntiferroNCRK23)
    AFf = true
}
solvertime = typ
```

Aunque apenas ha cambiado la estructura de los solvers con respecto a `aleste2` o `MuMax3`, por completitud, voy a escribir de forma explícita el funcionamiento de estos dos solvers.

### 4.1. RK23

Este método se llama Bogacki–Shampine [7] y es del tipo Runge-Kutta de orden tres que incluye una aproximación de orden dos con la que se calcula un paso temporal adaptativo. Los cálculos

que suceden en cada paso temporal son los siguientes:

$$\begin{aligned}
(k_{11}, k_{21}, k_{31}) &= \vec{\tau}(t_0; \vec{m}_{1,n}, \vec{m}_{2,n}, \vec{m}_{3,n}), \\
(k_{12}, k_{22}, k_{32}) &= \vec{\tau}(t_0 + 1/2 \cdot \Delta t; \vec{m}_{1,n} + 1/2 \cdot \gamma_1 k_{11}, \dots), \\
(k_{13}, k_{23}, k_{33}) &= \vec{\tau}(t_0 + 3/4 \cdot \Delta t; \vec{m}_{1,n} + 3/4 \cdot \gamma_1 k_{12}, \dots), \\
(\vec{m}_{1,n+1}, \vec{m}_{2,n+1}, \vec{m}_{3,n+1}) &= (\vec{m}_{1,n} + 2/9 \cdot \gamma_1 k_{11} + 1/3 \cdot \gamma_1 k_{12} + 4/9 \cdot \gamma_1 k_{13}, \dots), \\
(k_{14}, k_{24}, k_{34}) &= \vec{\tau}(t_0 + \Delta t; \vec{m}_{1,n+1}, \dots), \\
(\vec{z}_{1,n+1}, \vec{z}_{2,n+1}, \vec{z}_{3,n+1}) &= (\vec{z}_{1,n} + 7/24 \cdot \gamma_1 k_{11} + 1/4 \cdot \gamma_1 k_{12} + 1/3 \cdot \gamma_1 k_{13} + 1/8 \cdot \gamma_1 k_{14}, \dots).
\end{aligned}$$

Donde las menciones a  $\vec{\tau}(t; \vec{m}_i)$  en el código son llamadas a la función `torqueFnAFNC`,  $\vec{z}_i$  es una aproximación de segundo orden de  $\vec{m}_i$  y  $\gamma_i$  es el coeficiente giromagnético para cada subred. Los subíndices de  $k_{ij}$  son  $(i, j) = (\text{subred}, \text{orden})$  y el subíndice  $n$  se refiere al paso temporal, es decir, cuantos  $\Delta t$  han pasado desde la ejecución del programa, cuya duración es adaptativa. Para ajustar dicho paso temporal se comprueba si el error cometido es menor que el máximo permitido o si la duración del paso es menor de la mínima permitida. Ese error se calcula como

$$err = \max\{|\vec{m}_{i,n} - \vec{z}_{i,n}| \cdot \gamma_i\}.$$

No se van a entrar en los detalles de la adaptación del paso, para ello hay una función dedicada definida en `MuMax3` llamada `adaptDt()` que no se ha modificado.

## 4.2. RK4

También se ha implementado el método de Runge-Kutta de orden cuatro con paso adaptativo. Para cada paso temporal tenemos:

$$\begin{aligned}
(k_{11}, k_{21}, k_{31}) &= \vec{\tau}(t_0; \vec{m}_{1,n}, \vec{m}_{2,n}, \vec{m}_{3,n}), \\
(k_{12}, k_{22}, k_{32}) &= \vec{\tau}(t_0 + 1/2 \cdot \Delta t; \vec{m}_{1,n} + 1/2 \cdot \gamma_1 k_{11}, \dots), \\
(k_{13}, k_{23}, k_{33}) &= \vec{\tau}(t_0 + 1/2 \cdot \Delta t; \vec{m}_{1,n} + 1/2 \cdot \gamma_1 k_{12}, \dots), \\
(\vec{m}_{1,n+1}, \vec{m}_{2,n+1}, \vec{m}_{3,n+1}) &= (\vec{m}_{1,n} + \gamma_1 k_{13}, \dots), \\
(k_{14}, k_{24}, k_{34}) &= \vec{\tau}(t_0 + \Delta t; \vec{m}_{1,n+1}, \dots).
\end{aligned}$$

El código para adaptar el paso temporal es idéntico al caso anterior, solo que ahora el error se calcula de forma distinta

$$err = \max\{|k_{i,1} - k_{i,4}| \cdot \gamma_i\}.$$



## Problemas conocidos

1. Combinación de constantes de intercambio dan lugar a divisiones entre cero no previstas por las ecuaciones.
2. Simulaciones a  $T \neq 0$  generan ruido. Posible explicación: AFf flag en archivo run.
3. Falta por implementar DMIBulk.
4. Falta por implementar ecuación LLB.
5. NormalizeAFNC y MagnetizationAFNC son funciones duplicadas.

## Herramientas para tratar los datos

El código aquí descrito fue usado para la redacción de mi TFG. Para tratar los archivos obtenidos en las simulaciones, escribí algunos programas en **Python** que generan gráficas y ajustes de magnitudes relevantes. Estas herramientas se pueden encontrar en el mi repositorio [afnc.tools](#) junto a una explicación de su uso.

## Referencias

- [1] aleste2. *GPU-accelerated micromagnetic simulator, modified for antiferromagnets*. <https://github.com/aleste2/3>.
- [2] Arne Vansteenkiste et al. “The design and verification of MuMax3”. En: *AIP Advances* 4.10 (oct. de 2014), pág. 107133. ISSN: 2158-3226. DOI: [10.1063/1.4899186](https://doi.org/10.1063/1.4899186). eprint: [https://pubs.aip.org/aip/adv/article-pdf/doi/10.1063/1.4899186/12878560/107133\\_1\\_online.pdf](https://pubs.aip.org/aip/adv/article-pdf/doi/10.1063/1.4899186/12878560/107133_1_online.pdf). URL: <https://doi.org/10.1063/1.4899186>.
- [3] Víctor Costilla López. *GPU-accelerated micromagnetic simulator, modified for non collinear antiferromagnets*. <https://github.com/ribtaur/3>.
- [4] Luis Sánchez-Tejerina et al. “Dynamics of domain-wall motion driven by spin-orbit torque in antiferromagnets”. En: *Phys. Rev. B* 101 (1 2020), pág. 014433. DOI: [10.1103/PhysRevB.101.014433](https://doi.org/10.1103/PhysRevB.101.014433). URL: <https://link.aps.org/doi/10.1103/PhysRevB.101.014433>.
- [5] Luis Sánchez-Tejerina San José. “Modelización de nanodispositivos magnéticos con especial énfasis en los fenómenos de acoplamiento espin-órbita”. Tesis doct. 2018.
- [6] J. M. D. Coey. “Ferromagnetism and exchange”. En: *Magnetism and Magnetic Materials*. Cambridge University Press, 2010, 128–194.
- [7] *Bogacki–Shampine method*. Wikimedia Foundation. URL: [https://en.wikipedia.org/wiki/Bogacki%E2%80%93Shampine\\_method](https://en.wikipedia.org/wiki/Bogacki%E2%80%93Shampine_method).